

BOOKRUNNER (\$BKRN) — Backend

Scope: contracts, services, data, API, flows, milestones for the perp-market underwriting syndicate on Robinhood Chain (4663). Reads with Bookrunner_Overview.md. Stack: Bun · Hono · tRPC v11 · Drizzle · Postgres + Timescale · Redis + BullMQ · viem · Foundry · ERC-4337 session keys · Dwellir archive RPC. Non-upgradeable except `Book` logic library and venue adapters (UUPS, 48 h timelock). Every unverified address/ABI is marked VERIFY.

1. Architecture

```
Sponsor —charter (bond)→ MarketCharter → RiskCommittee (jury + 3 bonded) —a
Book { SeniorTranche (4626), JuniorTranche (4626), UnderwritingVault, MMMandate,
  |— Orderly path: IF account + MM account on venue (trade-only keys) ← bookrun
  |— In-house path: PoolEngine market (attested oracle, off-hours regime) ← boo
Risk service: limits, key revocation, cancels · Mark service: daily signed NAV +
```

2. Contracts (Foundry)

2.1 MarketCharter

```
struct Charter {
    bytes32 underlying;           // canonical Stock Token addr or index id
    uint8   venue;               // 0 = Orderly PerpAnything, 1 = in-house PoolEng
    uint8   oracle;              // 0 = Chainlink feed, 1 = attested multi-source
    bytes32 sessions;           // encoded trading sessions (RWA)
    uint128 ifTargetUsd;         // ≥ venue minimum (Orderly ≈ 25k USDC/symbol, VE
    uint128 mmInventoryUsd;
    Mandate mandate;            // see 2.4
    uint16  seniorHurdleBps;     // Senior's share of net fee flow before Junior r
    uint16  seniorCapBps;       // Senior max share of book capital, e.g. 7000
    uint32  subscriptionWindow; // seconds
    uint64  juniorNoticeSeconds; // 7d default; never a gate
    address sponsor;            // must hold ≥ 10% of Junior at close of window
}

function file(Charter calldata c) external payable;           // bonds $BKRN via s
function decide(uint id, bool ok, bytes32 juryCid) external onlyCommittee; // 2
```

2.2 Book (clone per charter) + tranches

- `SeniorTranche` , `JuniorTranche` : ERC-4626 over the book's NAV share; deposits only during the window (and later top-up rounds the sponsor opens); **withdrawals always honoured at the next daily mark** (Junior after notice). `pause()` blocks deposits only.
- `UnderwritingVault` : holds capital not yet deployed; `deployToVenue(amount)` → venue adapter; `recall(amount)` only to the vault.
- NAV per tranche computed by `Waterfall.split(bookNav)` : losses Junior → Senior; fee flow expenses → protocol carry (10%) → Senior hurdle share → Junior residual.

2.3 Venue adapters

- `OrderlyAdapter` (UUPS): creates the symbol under the builder account (off-chain API by the ops service; on-chain deposit to Orderly's RHC vault contract VERIFY), funds the per-symbol IF account, holds MM margin; **withdrawals only to** `UnderwritingVault`; daily taker-fee settlement swept into `RevenueRouter`.
- `PoolEngineAdapter` : lists a market on the in-house pool-vs-trader engine (Synthetix V3 lineage; attested TEE oracle; off-hours regime: index held → liquidate on margin only); IF and MM inventory live in the engine's market accounts; fees routed on each fill.

2.4 `MMMandate` (session-key validator for on-chain legs + policy mirror for the risk service)

```
struct Mandate {
    uint128 maxInventoryUsd; int16 maxSkewBps; uint16 minQuoteWidthBps; uint16 maxH
    uint16 hedgeRatioMinBps; uint16 hedgeRatioMaxBps; bool noNewRiskOffHours;
    int16 killAtDrawdownBps; // book-level, e.g. -800 → cancel all, flatten, re
    bytes32 hedgeAllowRoot; // (asset, venue) pairs for hedge legs
}
```

On-chain: validates bookrunner userOps (hedge legs in canonical Stock Tokens via Uniswap v4/v3, inventory moves between vault and venue) against the mandate; reverts otherwise. Off-chain quoting is covered by the risk service (§3) with the venue's own limits (isolated margin, position caps) as the hard stop.

2.5 `RevenueRouter` + `Waterfall`

Receives venue settlements and engine fees; subtracts expenses (oracle, keeper gas, charter costs) → 10% to `BkrnFeeRouter` (50% buyback-to-stakers, 50% syndicate backstop) → Senior hurdle → Junior residual; emits `Distributed(bookId, period, amounts[])`. Losses realised on mark: `Waterfall.absorb(loss)` Junior first.

2.6 RiskCommittee

Three bonded members (staked \$BKRN, slashable on proven negligence by governance) + required jury verdict CID; 2-of-3 to approve; quorum changes via timelock.

2.7 MarkRegistry

```
commit(bookId, periodEnd, navUsd, inventoryRoot, pnlJson hash, receiptsRoot, sig)
```

by the mark signer; anyone can verify tranche NAVs from the mark; tranches read the latest mark for redemptions.

2.8 BkrnStaking , BkrnFeeRouter , Backstop

Tiers (sponsor, committee, bookrunner inventory tiers); carry split; `Backstop.cover(bookId, shortfall)` for Senior after Junior = 0, up to balance.

2.9 Red-team checks in tests

Waterfall ordering under partial losses · redemption always honoured (invariant: `redeem` never reverts for lack of permission, only for lack of liquidity after recall, which the mark service must prevent) · mandate escalation · key revocation race · off-hours quoting blocked · oracle staleness → engine pause (new risk only) · Stock Tokens not borrowable (hedges are long-spot or perp-based only) · float caps · multiplier double-apply on hedge valuation.

3. Services

Service	Role
<code>charter</code>	Charter intake, jury request, committee workflow, bond/fee handling
<code>ops-venue</code>	Orderly builder API: symbol creation, oracle config, IF funding, settlements (REST/WS; trade-only + builder keys split), VERIFY endpoints
<code>bookrunner-agent</code>	One process per book: quoting model (inventory-aware Avellaneda-style with mandate constraints), hedge planner, kill logic; venue trade-only key; on-chain session key for hedge legs
<code>risk</code>	Live limit utilisation; breach → cancel-all + revoke venue key + on-chain <code>revoke(deskKey)</code> ; off-hours flags from feeds; drawdown state
<code>oracle</code>	Attested multi-source TEE oracle for in-house markets; publishes signed prices + hold flags
	Daily NAV per book: venue equity + vault + hedge inventory (multiplier-

mark	aware) → signed <code>MarkRegistry.commit</code> ; receipts root
waterfall	Settlement sweeps → <code>RevenueRouter</code> ; distribution events → tranche NAV updates
receipts	Hourly Merkle root over quotes, fills, hedges, decisions
api	trRPC/REST, dashboards, webhooks, read-only MCP (<code>book_nav</code> , <code>book_limits</code> , <code>charters</code>)

4. Data

```

charters(id, sponsor, struct_json, status, jury_cid, decided_at, bond_tx)
books(id, charter_id, senior_addr, junior_addr, vault_addr, venue, symbol, create
subscriptions(book_id, tranche, wallet, shares, ts)
redemptions(id, book_id, tranche, wallet, shares, notice_at, honoured_mark_id)
venue_accounts(book_id, kind, account_id, key_prefix, status)
quotes(book_id, ts, bid, ask, size, inventory_usd, skew_bps)           -- hyper
fills(book_id, ts, side, qty, px, fee_usd, venue_trade_id)
hedges(book_id, ts, asset, qty_raw, px, mult, tx_hash)
limits(book_id, ts, inventory_util, skew_util, hedge_ratio, drawdown_bps, state)
marks(id, book_id, period_end, nav_usd, senior_nav, junior_nav, pnl_json, receipt
settlements(book_id, ts, source, gross_usd, expenses_usd, carry_usd, senior_usd,
committee(member, bond, votes_json)

```

5. API

```

charter.file/get/list/decide · book.list/get/nav/limits/marks ·
tranche.subscribe/redeem(notice) · agent.register/revoke(bookId, key) ·
risk.state(bookId) · settlements.list · receipts.root/proof · webhooks:
charter.decided, limit.breached, kill.executed, mark.committed,
distribution.paid.

```

6. Flows

1. **Charter → book:** sponsor files (bond + fee) → jury verdict → committee 2-of-3 → `BookFactory.create` → subscription window opens.
2. **Subscription:** deposits into Senior/Junior with per-wallet cap and `seniorCapBps` ; window closes → sponsor's $\geq 10\%$ Junior check → `deployToVenue` (IF + MM inventory) → symbol goes live.

3. **Quote/hedge:** agent streams quotes within mandate; hedge planner keeps hedge ratio in band using long Stock Token spot (v4) and/or offsetting perps on allow-listed venues; off-hours → reduce-only.
4. **Breach/kill:** risk detects utilisation > 100% or drawdown ≤ kill → cancel-all → flatten within mandate → revoke keys → sponsor notified → committee may re-mandate.
5. **Daily:** venue settlement swept → waterfall → mark committed → tranche NAVs update → queued redemptions honoured.
6. **Wind-down:** sponsor or committee files `retire(bookId)` → quoting stops → inventory flattened → IF recalled per venue rules → final mark → full redemption.

7. Fees

10% protocol carry on net fee flow (before Senior) → 50% buyback-to-stakers / 50% backstop. Flat charter fee in USDC (refundable on rejection). No fee on capital.

8. Build order (Claude-Code-sized; three lanes)

Lane A — contracts: A1 `MarketCharter` + `RiskCommittee` + bonds · A2 tranches (4626) + `Waterfall` with partial-loss fuzz · A3 `UnderwritingVault` + recall rules · A4 `MMMandate` validator + hedge-leg executor on fork (v4/v3) · A5 `OrderlyAdapter` (deposit/withdraw to vault only; VERIFY contracts) · A6 `PoolEngineAdapter` on the in-house engine · A7 `RevenueRouter` + fee router + `Backstop` · A8 `MarkRegistry` · A9 `BookFactory` · A10 invariants (redemption never permission-gated; waterfall conservation; mandate bounds).

Lane B — services: B1 schema · B2 oracle service (attested, multi-source, hold flags) · B3 ops-venue (Orderly builder flow, VERIFY) · B4 bookrunner-agent v1 (rule-based quoting; model-assisted sizing behind the same limits) · B5 risk + kill + key revocation · B6 mark + receipts · B7 waterfall sweeps · B8 api/dashboards/webhooks/MCP · B9 charter workflow + jury.

Lane C — launch: C1 three studio books on fork (NVDA, TSLA, Stock-Token index) · C2 bounty scope (mandate, waterfall, adapters) · C3 book dashboards (NAV, limits, marks, fills) · C4 mainnet: charters filed, windows open, first marks.

9. Honest limits

Venue margin and liquidation are the venue's; exposure is bounded by deposits · venue quoting is monitored-and-revoked, not contract-validated; venue limits are the hard stop · Senior is last-loss, not no-loss · builder-oracle markets carry our oracle's risk · Orderly specifics re-verified at build · no US marketing of stock-perp books.

